

| Introduction to SV

Naresh Kumar Yadav

Agenda

Introduction to SV

- History and Evolution
- Benefits of SV
- Overview

Data Types

- **User defined types**
- **Enumeration**
- **Parameterized types**

Arrays & Structures

- **Single/Multi-dimensional**
- **Dynamic Arrays**
- **Associative Arrays**
- **Queues**
- **Structures/Unions**

History and evolution

Verilog Origins & Standardization (1984–1995)

1984: Phil Moorby developed the Verilog language and simulator at Gateway Design Automation.

1989: Gateway was acquired by Cadence Design Systems.

1990: Cadence placed Verilog in the public domain, leading to the formation of Open Verilog International (OVI).

1995: Verilog was officially standardized by the IEEE as IEEE Std 1364-1995 (Verilog-95)



Verilog-2001 & Enhanced Syntax (2001–2005)

2001: Recognizing the limitations of Verilog-95, the IEEE released IEEE Std 1364-2001, commonly known as Verilog-2001. This update introduced massive improvements, including multi-dimensional arrays, named port connections, and enhanced syntax for signed arithmetic and generate statements.

2005: A minor update was published as IEEE Std 1364-2005, providing minor clarifications



The Birth of SystemVerilog (2002–2005)

As integrated circuits grew in complexity, traditional Verilog testbenches struggled to thoroughly verify modern architectures. [1]

2002: The startup Co-Design Automation donated its "Superlog" language to Accellera, laying the framework for advanced design and C/C++ integration.

2002: Synopsys donated "OpenVera," establishing the foundational object-oriented syntax for modern, constrained-random verification.

2005: Accellera's work culminated in IEEE Std 1800-2005, officially establishing SystemVerilog as a distinct extension to Verilog. [1, 2, 3]

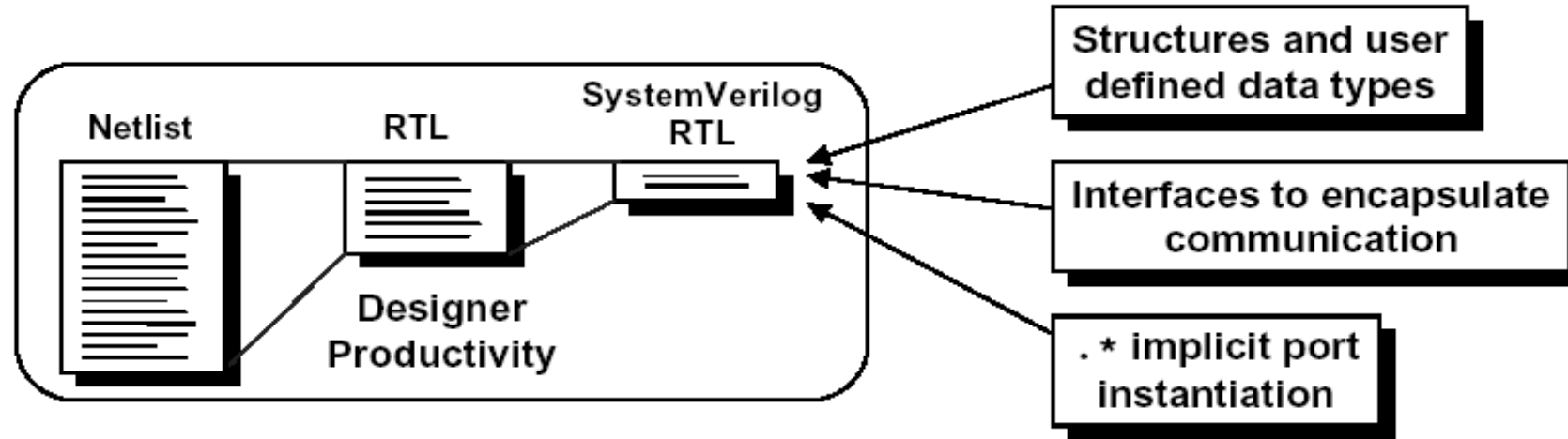


Merged Standard & Evolution (2009–Present)

2009: The IEEE merged the base Verilog standard and SystemVerilog extensions into a single unified language document, IEEE Std 1800-2009. [1]

2012, 2017 & 2023: The IEEE released updated iterations—such as IEEE Std 1800-2012, [IEEE Std 1800-2017](#), and IEEE Std 1800-2023—which refined assertion constructs, improved simulator performance, and expanded object-oriented capabilities for hardware verification

System Verilog Benefits

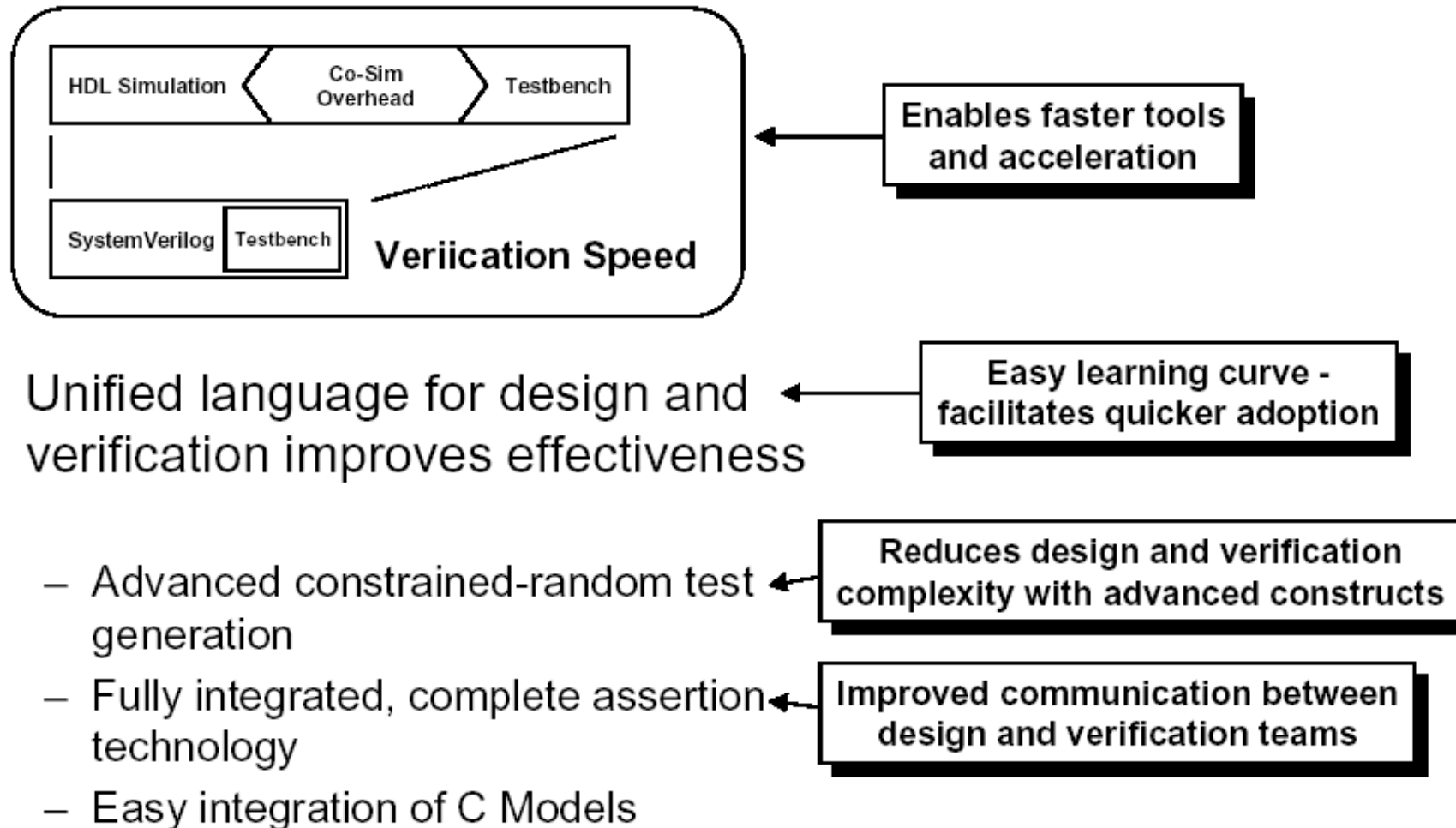


- 2X – 5X less code to capture the same functionality
 - Less code → Fewer bugs
 - Easier to interpret and communicate among teams
- Evolutionary: Reduces learning curve

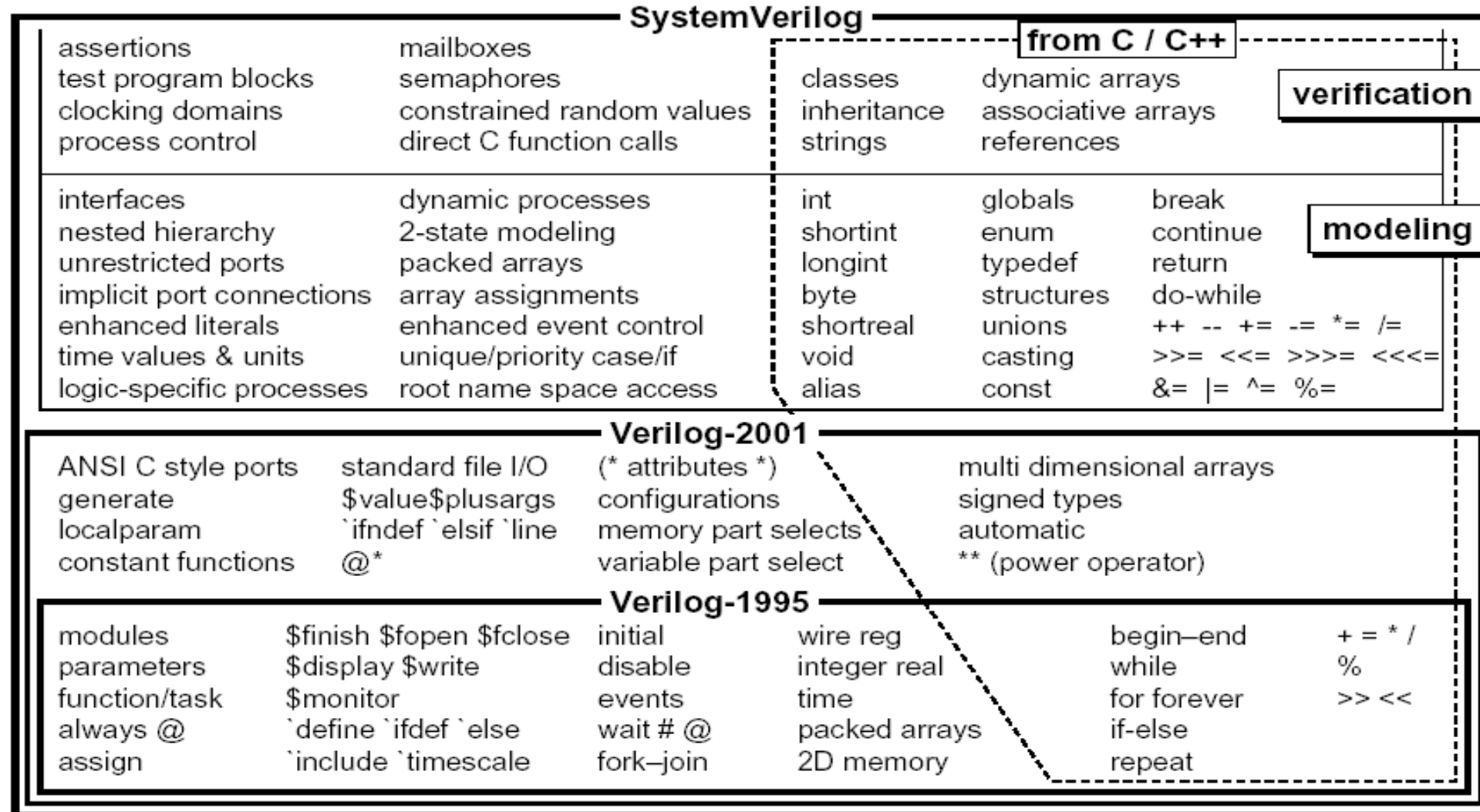
New constructs to help
*eliminate simulation and
synthesis mismatches*

Less code &
still synthesizable

System Verilog Benefits



SV Overview



SV Data Types

SystemVerilog Data Types : Value Set

The SystemVerilog value set consists of the following four basic values:

- 0—represents a logic zero or a false condition
- 1—represents a logic one or a true condition
- x—represents an unknown logic value
- z—represents a high-impedance state

4 state types: can store all 4 logic values 0,1,x,z

2 state types: can store only 0,1 as value.

SystemVerilog Data Types

Void Type

- void: represents nonexistent data.
- This type can be specified as the return type of functions to indicate no return value

Integer data types

<code>shortint</code>	2-state data type, 16-bit signed integer
<code>int</code>	2-state data type, 32-bit signed integer
<code>longint</code>	2-state data type, 64-bit signed integer
<code>byte</code>	2-state data type, 8-bit signed integer or ASCII character
<code>bit</code>	2-state data type, user-defined vector size, unsigned
<code>logic</code>	4-state data type, user-defined vector size, unsigned
<code>reg</code>	4-state data type, user-defined vector size, unsigned
<code>integer</code>	4-state data type, 32-bit signed integer
<code>time</code>	4-state data type, 64-bit unsigned integer

Prompt:

Write SV code to show the usage of reg, wire, bit and logic.

SystemVerilog DataTypes: real type

Real Types:

- **real**: 2 state, 64-bit
- **sortreal**: 2 state, 32-bit
- **realtime**: same as real used for time
- assignment to integer variable:
 - converted to integers by rounding the real number to the nearest integer, rather than by truncating it.
 - `int a = 7.6;` a is 8 and **not 7**
- real variables are **prohibited** in the following cases:
 - Edge event controls (posedge, negedge, edge) applied to real variables
 - Bit-select or part-select references of variables declared as real
 - Real number index expressions of bit-select or part-select references of vectors

Prompt:

Write example SV code to show the usage of real and realtime.

SystemVerilog DataTypes: Strings

String data type is a variable size array of characters, indexed from 0 to N-1 (N is array length)

- Every element of the array is also a string

```
string st1, st2;  
st1 = ""; // null string  
st2 = "abc"; // assigned from a string literal  
st1 = { st1, st2 }; // concatenation, st1 becomes "abc"  
st2[0:1] = "et"; // st2 becomes "etc"
```

Prompt:

Tell me an real world example in SV where string manipulation is required.

Supports relational operators (`==`, `!=`, `<`, `<=`, `>`, `>=`), concatenation (`{ }`) and replication (`{n{ }}`)

- e.g. (a<b) is true because a precedes b alphabetically

By means of the `.` operator, strings support special methods: `str.method(...)`

- `len()`, `putc()`, `getc()`, `toupper()`, `tolower()`, `compare()`, `icompare()`, `substr()`, `atoi()`, `atohex()`, `atooct()`, `atobin()`, `atoreal()`, `itoa()`, `hextoa()`, `octtoa()`, `bintoa()` and `realtoa()`

Literals

Integer literal constants can 16'sbz be specified in decimal, hexadecimal, octal, or binary format.

- Example 1: Unsized literal constant numbers
 - 659 // is a decimal number
 - 'h 837FF // is a hexadecimal number
 - 'o7460 // is an octal number
 - 4af // is illegal (hexadecimal format requires 'h)
- Example 2: Sized literal constant numbers
 - 4'b1001 // is a 4-bit binary number
 - 5 'D 3 // is a 5-bit decimal number
 - 3'b01x // is a 3-bit number with the least significant bit unknown
 - 12'hx // is a 12-bit unknown number
 - 16'hz // is a 16-bit high-impedance number
- Example 3: Using sign with literal constant numbers
 - 8 'd -6 // this is **illegal syntax**
 - -8 'd 6 // this defines the two's-complement of 6, held in 8 bits—equivalent to -(8'd 6)
 - **4 'shf // this denotes the 4-bit number '1111', to be interpreted as a two's-complement number, or '-1'. This is equivalent to -4'h 1**
 - -4 'sd15 // this is equivalent to -(-4'd 1), or '0001'
 - 16'sd? // the same as 16'sbz
- SystemVerilog allows easy specification of un-sized literal values with the (') apostrophe
 - '0, '1, 'x, 'X, 'z, 'Z // Notice, no base specifier needed
 - E.g.: reg [23:0] a = '1; // All bits of a are set to "1". a = 24'hFFFFFF
- Underscores in literal constant numbers
 - 27_195_000 // unsized decimal 27195000
 - 16'b0011_0101_0001_1111 // 16-bit binary number
 - 32 'h 12ab_f001 // 32-bit hexadecimal number

Literals

Real literals

- Real numbers can be specified in either decimal notation (for example, 14.72) or in scientific notation
- Example: 39e8, which indicates 39 multiplied by 10 to the eighth power).
 - 1.2, 0.1, 2394.26331
 - 1.2E12 (the exponent symbol can be e or E)
 - 1.30e-2, 0.1e-0, 23E10, 29E-2, 236.123_763_e-12 (underscores are ignored)
- Real numbers expressed with a decimal point shall have at least one digit on each side of the decimal point.
 - .12, 9., 4.E3, .2e-7 //invalid

Time literals

- Time is written in integer or fixed-point format, followed without a space by a time unit (fs ps ns us ms s).
 - realtime t1 = 2.1ns; time t2 = 40ps

String literal

- A string literal is a sequence of characters enclosed by double quotes ("").
- Nonprinting and other special characters are preceded with a backslash (\)
 - string s = "hello"
 - \$display("Humpty Dumpty sat on a wall.\n");

User defined types

SystemVerilog's data types can be extended with user-defined types using `typedef`

- Syntax: `typedef` <base_data_type> <type_identifier>
- Example:

```
typedef int inch ;// inch becomes a new type
```

```
inch foot = 12, yard = 36;    // these are 2 new variables of type 'inch'
```

Enumerated Types

Syntax: `enum` [enum_base_type] { enum_name_declaration {,enum_name_declaration} } [var_name]

- enum_base_type: default is `int`

Enumeration is a useful way of defining abstract variables.

- `enum {red, green, yellow} traf_lite;`

Values can be cast to integer types and auto-incremented

- `enum { a=5, b, c} vars1,vars2; // b=6, c=7`

Default assigned values start at zero

```
      0      1      2
enum {red, green, yellow} lite;
```

A sized constant can be used to set size of the type

- `enum bit [3:0] {bronze=4'h3, silver, gold} medal; // All medal members are (must be) 4-bits`

An enumerated name with x or z assignments assigned to an enum with no explicit data type or an explicit : **ERROR**

- `enum bit [1:0] {IDLE, XX='x, S1=2'b01, S2=2'b10} state, next;`

An enum declaration of a 4-state type, such as integer, that includes one or more names with x or z assignments is **permitted.**

- `enum integer {IDLE, XX='x, S1='b01, S2='b10} state, next`
- `enum integer {IDLE, XX='x, S1, S2} state, next ; what happens in this case ??`

Enumerated Types

Define a new type

- `typedef enum {TRUE, FALSE} boolean;` // *boolean is NOT a System Verilog type*
- `boolean` statement; → statement is of Boolean type now and can take only TRUE and FALSE as values
- Example:

```
module enum_types;
typedef enum {red, green, blue, yellow, white, black} colors ;
colors my_colors;
initial begin
    my_colors = green;
// my_colors = 1;           // invalid, needs to typecast
// my_colors = colors'(2);  // cast as blue
    case (my_colors)
        red: $display ("my_color is red");
        blue: $display ("my_color is blue");
        default: $display ("my_color is not blue or red");
    endcase
end
endmodule
```

Output: my_color is not blue or red

Enumeration Methods

SystemVerilog provides methods to allow easy manipulation of enumerated types

- function enum `first()`
 - returns the value of the first member of the enumeration enum.
- function enum `last()`
 - returns the value of the last member of the enumeration enum.
- function enum `next(int unsigned N = 1)`
 - returns the Nth next enumeration value (default is the next one) starting from the current value of the given variable.
 - Note: `next()` "wraps" to the first value when applied to the last
- function enum `prev(int unsigned N = 1)`
 - returns the Nth previous enumeration value (default is the previous one) starting from the current value of the given variable.
 - Note: `prev()` "wraps" to the last value when applied to the first
- function int `num()`
 - returns the number of elements in the given enumeration.
- function string `name()`
 - returns the string representation of the given enumeration value

Prompt:
How can we iterate over all values of enum variable?

Constants

- **Constants are named data types that never change**
- **SystemVerilog provides three elaboration-time constants**
 - parameter constants : **parameter**, **localparam**, and **specparam**
 - can be initialized with a literal.
 - localparam byte colon1 = ":" ;
 - specparam delay = 10 ; // specparams are used for specify blocks
 - parameter logic flag = 1 ;

```
module fifo #(
    parameter int DEPTH = 16,
    parameter int WIDTH = 8
);
    localparam int ADDR_W = $clog2(DEPTH); // derived —
    cannot be overridden
    localparam int WORDS = DEPTH * WIDTH; // also derived
```

parameter	localparam
Used when user want to configure elaboration time	Used when fixed internal constant is required
Inside module, interface, class, program, package	Inside module, interface, class, program, package
Can be changed during instantiation fifo #(32,16) my_fifo();	Cannot be overridden

constants: value and type parameter

Value parameter

- parameter msb = 7; // defines msb as a constant value 7
- parameter e = 25, f = 9; // defines two constant numbers
- parameter r = 5.7; // declares r as a real parameter
- parameter byte_size = 8, byte_mask = byte_size - 1;
- parameter average_delay = (r + f) / 2;
- parameter signed [3:0] mux_selector = 0;
- parameter real r1 = 3.5e17;
- parameter p1 = 13'h7e;
- parameter [31:0] dec_const = 1'b1; // value converted to 32 bits
- parameter newconst = 3'h4; // implied range of [2:0]
- parameter newconst = 4; // implied range of at least [31:0]
- parameter logic [31:0] P1 [3:0] = '{ 1, 2, 3, 4 }; // unpacked array

type parameter

- SystemVerilog extends Verilog parameters to support types

```
module ma #(parameter p1 = 1, parameter type p2 = shortint)
    (input logic [p1:0] i, output logic [p1:0] o);
    p2 j = 0; // type of j is set by a parameter, (shortint unless redefined)
    ...
    ...
endmodule

module mb;
    logic [3:0] i,o;
    ma #(.p1(3), .p2(int)) u1(i,o); //redefines p2 to a type of int
endmodule
```

Constants

- **SystemVerilog also provides a run-time constant: `const`**
- **Initialize once and then read only**
 - `const a = 10; //run time assignment.`
 - `a = 20; //Error`
 - `const logic option = a.b.c; //OK`
- **An automatic constant declared with the `const` keyword can be set to any expression that would be legal without the `const` keyword.**
- **An instance of a class (an object handle) can also be declared with the `const` keyword.**
 - `const class_name object = new(5,3);`
 - the object acts like a variable that cannot be written. The members of the object can be written (except for those members that are declared `const`).

Casting : static (compile time)

- Syntax: <type>'(value|variable)
- Convert between data types with the cast

```
reg [31:0] a ,b;  
typedef int mytype;  
mytype green;  
  
int' (2.0*3.0);    // Cast result to integer  
10' (a+b); // implies a number of bits (10 here)  
signed' (a);    // works for sign-changing too
```

User defined types may also be cast:

```
green = mytype' (b);
```

The following Verilog functions are still supported:

```
$itor(), $rtoi(), $bitstoreal(), $realtobits(), $signed(), $unsigned()
```

Casting: dynamic (run time)

- **\$cast**: system task can be used to assign values to variables that might not ordinarily be valid because of differing data type
- **\$cast can be called as either a task or a function.**
 - **function** int \$cast(data_type dest_variable, data_type source_expression);
 - **task** \$cast(data_type dest_variable, data_type source_expression);
- **Both forms attempt to assign source_expression to dest_variable**
 - **Function**: If assign is successful return 1, else return 0 and leave dest_variable unchanged
 - **Task**: If assign is not successful, trigger a runtime error and leave dest_variable unchanged

```
typedef enum { red, green, blue, yellow, white, black } Colors;  
Colors col;  
$cast( col, 2 + 3 ); //task
```

This code assigns 5 (or black) to col. Without \$cast, this assignment is illegal
-OR-

Use the function form of \$cast to check if the assignment will succeed:

```
if ( ! $cast( col, 24 ) )  
    $display( "Error in cast" );
```

Static/Dynamic Variables

- **Static (Verilog 1995)**

- Memory-allocation / initialization once, at compile time
- Exists for the entire time of block execution

Block can be compilation unit,
module, interface, program
block, function, task

- **Automatic (Verilog 2001)**

- Stack based
- Reallocated / initialized each time a block/task/function is entered
- Supports recursion in blocks, tasks and functions

- **SystemVerilog is backward compatible with both Verilog 1995 and 2001.**

Variables Scope and Lifetime

- **Scope: visibility of variable**
- **Lifetime: time duration for which variable has occupied memory.**
- **Variables declared outside a module, program, interface, checker, task, or function**
 - are local to the compilation unit (global scope)
 - have a static lifetime (exist for the whole simulation).
- **Variables declared inside a module, interface, program, or checker, but outside a task, process, or function**
 - are local in scope and have a static lifetime
- **Variables declared inside a static task, function, or block**
 - are local in scope and default to a static lifetime
 - Declared with **automatic**: have the lifetime of the call or block and are initialized on each entry to the call
- **Tasks and functions declared as automatic:** declared variables local in scope and automatic
- Within a module, interface, package, or program: **default lifetime of function and task is static**

Variables Scope and Lifetime ...

- Explicit static keyword shall be required when an initialization value is specified as part of a static variable's declaration
- The static keyword shall be optional where it would not be legal to declare the variables as automatic

```
int g_st0; //static and global
module msl;
  int st0; // static
  initial begin
    int st1; // static
    static int st2; // static
    automatic int auto1; // automatic
  end

  task automatic t1();
    int auto2; // automatic
    static int st3; // static
    automatic int auto3; // automatic
  endtask
endmodule
```

```
module top_legal;
  int svar1 = 1; // static keyword optional
  initial begin
    for (int i=0; i<3; i++) begin
      automatic int loop3 = 0; // executes every loop
      for (int j=0; j<3; j++) begin
        loop3++;
        $display(loop3);
      end
    end // prints 1 2 3 1 2 3 1 2 3
    for (int i=0; i<3; i++) begin
      static int loop2 = 0; // executes once at time zero
      for (int j=0; j<3; j++) begin
        loop2++;
        $display(loop2);
      end
    end // prints 1 2 3 4 5 6 7 8 9
  end
endmodule : top_legal
```

Variable scope and lifetime ...

- It is permissible to hierarchically reference any static variable unless the variable is declared inside an unnamed block
- Arguments of task and functions are automatic unless specified static
 - task t (int a); // a is automatic

```
module top_check; // compile check
  initial begin
    int svar2 = 2; // static/automatic needed to show intent
    for (int i=0; i<3; i++) begin
      int loop3 = 0;
      for (int i=0; i<3; i++) begin
        loop3++;
        $display(loop3);
      end
    end
  end
endmodule : top_check
```

Prompt:
Check this code for compilation errors
Fix the code and compile it

Aggregate Data Types

- ❑ Packed/unpacked arrays, dynamic arrays, associative arrays, and queues
- ❑ Array query and manipulation methods
- ❑ Structure definitions and usage
- ❑ Union definitions and usage

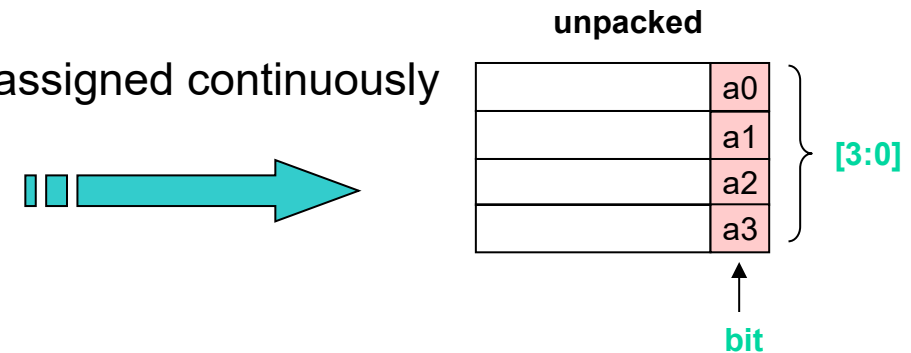
Array Data types

Packed and Unpacked: to refer to how the data is stored in memory

- **Unpacked**

- Can be made of any datatype
- Only access a single element at a time
 - Although whole arrays may be copied
- One element may be assigned procedurally while another is assigned continuously
- Specified as a range or size (`int mem [256:1]`)

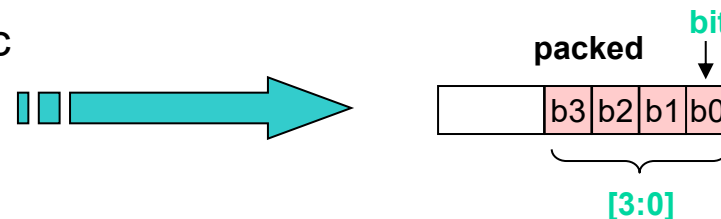
`bit a [3];` // implies `a[0:3]` and not `a[3:0]`



- **Packed**

- can be made of only the single bit data types (bit, logic, reg), enumerated types, and recursively other packed arrays and packed structures.
- Access whole array or slice as a vector
- All elements must be assigned identically
- Compatible with \$monitor/\$display etc.
- Allows arbitrary length integers/arithmetic

`bit [3:0] b;` // packed array of bits



Multidimensional Arrays

Array of arrays: just like Verilog

`logic [7:0] mem_a [0:255];` // declares a memory array of 256 8-bit elements. The array indices are 0 to 255

`mem_a[5][3] = 0;` // Write to bit 3 of byte at address 5 to 0

`data = mem_a[addr];` // Read word at address indexed by addr

- Dimensions declared following the type and before the name vary more rapidly than the dimensions following the name.

`bit [1:10] v1 [1:5];` // 1 to 10 varies most rapidly

- In a list of dimensions, the rightmost one varies most rapidly. packed dimension varies more rapidly than an unpacked

`bit [1:5] [1:6] v4 [1:7] [1:8];` // 1 to 6 varies most rapidly, followed by 1 to 5, then 1 to 8 and then 1 to 7

Sub-arrays: an array that is an element of another array

```
int A[2][3][4], B[2][3][4], C[5][4];
...
A[0][2] = B[1][1]; // assign a subarray composed of four ints
A[1] = B[0];       // assign a subarray composed of three arrays of
                   // four ints each
A = B;             // assign an entire array
A[0][1] = C[4];     // assign compatible subarray of four ints
```

Array Operations

Packed and unpacked

```
bit[7:0] PA, PB ;
int var;
initial begin
    var = 3;
    PA = PB;          // Read/write
    PA[7:4] = 'hA;    // Read/write of a slice
    PA[var -:4] = PA[var+1 +:4]; // Read/write of a variable slice (Verilog 2k syntax equiv: PA[3:0] = PA[7:4];
end
```

Only for Packed

```
bit[3:0][7:0] MPA;
initial begin
    MPA = 32'hdeadbeef; // Assignment from an integer
    MPA = MPA+1;        // Treatment as an integer in an expression
end
```

Multi-dimensional array examples

- The packed dimensions describe how the data is arranged (or packed)
- The unpacked dimensions describe how we map this data to a multi-dimension address

`bit [3:0] [7:0] aa ;`

`// a single, packed 4-byte data word (4x8 = 32 bits)`

byte3

byte2

byte1

byte0

aa

`aa[0] = aa[0] + 1; // byte increment`

`bit [3:0] [7:0] bb [1:0];`

`// 2-deep array of packed 4-byte data words`

byte3

byte2

byte1

byte0

bb[1]

byte3

byte2

byte1

byte0

bb[0]

`bb[1] = bb[0]; // word assignment`

byte3

byte2

byte1

byte0

bb[1]

byte3

byte2

byte1

byte0

bb[0]

byte3

byte2

byte1

byte0

bb[1]

byte3

byte2

byte1

byte0

bb[0]

`bb[1][2][3:0] = bb[0][1][7:4]; // nibble copy`

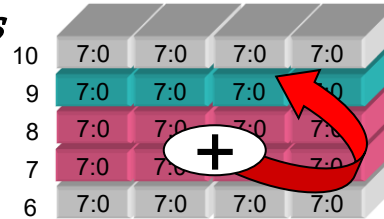
Multi-dimensional array examples ...

```
bit [3:0] [7:0] bb [10:1];    // array of 10, 4-byte words
```

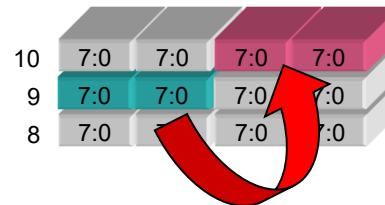
To access: start with unpacked dimensions proceeding left to right then continue with the packed dimensions, also proceeding left to right



```
bb[9] = bb[8] + bb[7];    // add 2, 4-byte words
```



```
bb[10][1:0] = bb [9][3:2];    // copy 2 MS bytes  
// from word 9 to word 10 (LS bytes)
```



Array Literals

Assigning literal values to SV arrays:

```
int k [1:3][1:4] = {default: 5}; // All elements "5"
```

For more control, consider the dimensions of the array and use { } to match those dimensions exactly

```
int k [1:3][1:4] = {{1,2,3,4},{5,6,7,8},{9,10,11,12}}; // 3 groups of 4
```

```
int m [1:2][1:3] = {{0,1,2},{3{4}}}; // 2 groups of 3
```

```
char a [1:12] = {"H","e","l","l","o","J","W","o","r","l","d","\n"}; // a group of 12 char's
```

Dynamic Arrays

- Unpacked array whose size can be set or changed at run time
- Default size of an uninitialized dynamic array is zero
- **Syntax:** `data_type array_name[] ;` //declares a dynamic array *array_name* of type *data_type*
- Size is set by using `new[]` constructor

```
bit [3:0] nibble[ ]; // Dynamic array of 4-bit vectors
integer mem[ ];      // Dynamic array of integers
int data[ ];          // Declare the dynamic array
data = new [256];     // Create a 256-element array
int addr[ ] = new [100]; // Create a 100-element array
addr = new [200] (addr); // Create a 200-element array preserving previous values in lower 100 addresses
int src[3], dest1[], dest2[];
src = '{2, 3, 4};
dest1 = new [2] (src); // dest1's elements are {2, 3}.
dest2 = new [4] (src); // dest2's elements are {2, 3, 4, 0}.
```

Dynamic array Methods

function int **size()**

- Returns the current size of the array

```
int addr[ ] = new[256];  
int j = addr.size();    // j = 256
```

function void **delete()**

- Empties array contents and zero-sizes it

```
int addr[ ] = new[256];  
addr.delete();
```

Multidimensional dynamic array

Dynamic arrays can be used with static arrays or alone to create multidimensional arrays

```
int arr1 [][2][3] = new [4]; // arr1 sized to length 4; elements are
                               // fixed-size arrays and so do not require initializing
int arr2 [][] = new [4];      // arr2 sized to length 4; dynamic subarrays remain unsized and uninitialized
int arr3 [1][2][] = new [4]; // Error – arr3 is not a dynamic array, though it contains dynamic subarrays
```

```
int arr[2][][]; //arr[0][][] and arr[1][][]
arr[0] = new [4]; // dynamic subarray arr[0] sized to length 4 i.e. arr[0][<0,1,2,3>][]
```

```
arr[0][0] = new [2]; // legal, arr[0][n] created above for n = 0..3 i.e. arr[0][0][0] and ar[0][0][1]
```

```
arr[1][0] = new [2]; // illegal, arr[1] not initialized so arr[1][0] does not exist
```

```
arr[0][] = new [2]; // illegal, syntax error - dimension without subscript on left hand side
```

```
arr[0][1][1] = new[2]; // illegal, arr[0][1][1] is an int, not a dynamic array
```

Associative arrays

- **Associative arrays (sometimes called indexed arrays)**
 - Support situations where data set size is totally unpredictable and elements may be added or removed individually to grow/shrink the array
 - Implemented as a look up table and so require an index.

Syntax:

`data_type array_id [ index_type ];` *// index type is the datatype to use as index*
 // examples include string, int, class, struct

```
bit i_array[*];           // associative array of bits (unspecified index)
                           // unspecified index (*) implies any integral value

bit [7:0] age [string];   // associative array of 8-bit vectors, indexed by string
```

```
string tom = "tom";
age [tom] = 21;
age ["joe"] = 32;
$display("%s is %d years of age ", tom, age[tom], "[%0d ages available]",age.num());
```

Associative array methods

several built-in methods are provided

- `num()` and `size()`: return the number of entries in the associative array. If empty, return 0

```
int imem[int];  
imem[ 3 ] = 1;  
imem[ 16'hffff ] = 2;  
imem[ 4'b1000 ] = 3;  
$display( "%0d entries\n", imem.num ); // prints "3 entries"
```

- `delete(index)`

- If the index is specified, then removes the entry at the specified index else removes all elements

```
int map[ string ];  
map[ "hello" ] = 1;  
map[ "sad" ] = 2;  
map[ "world" ] = 3;  
map.delete( "sad" ); // remove entry whose index is "sad" from "map"  
map.delete; // remove all entries from the associative array "map"
```

Associative array methods

- **exists**(index)
 - checks whether an element exists at the specified index within the given array.
 - returns 1 if the element exists; otherwise

```
if ( map.exists( "hello" ))
    map[ "hello" ] += 1;
else
    map[ "hello" ] = 0;e, it returns 0.
```
- **first**(index)
 - assigns to the given index variable the value of the first (smallest) index in the associative array.
 - returns 0 if the array is empty; else return 1

```
string s;
if ( map.first( s ) )
    $display( "First entry is : map[ %s ] = %0d\n", s, map[s] );
```
- **last**(index)
 - assigns to the given index variable the value of the last (largest) index in the associative array.
 - returns 0 if the array is empty; else return 1

```
string s;
if ( map.last( s ) )
    $display( "Last entry is : map[ %s ] = %0d\n", s, map[s] );
```

Associative array methods

- **next**(index)

- finds the smallest allowed index whose value is greater than the given index argument
- wildcard index type arrays not allowed.

```
string s;  
//iterate over complete array  
if ( map.first( s ) )  
do  
    $display( "%s : %d\n", s, map[ s ] );  
while ( map.next( s ) );
```

- **prev**(index)

- finds the largest index whose value is smaller than the given index argument

```
string s;  
//iterate over complete array in reverse direction  
if ( map.last( s ) )  
do  
    $display( "%s : %d\n", s, map[ s ] );  
while ( map.prev( s ) );
```


Associative array literals

- **Associative array literals use the '{index:value} syntax with an optional default index**
- **If a default value is specified, then reading a nonexistent element shall yield the specified default value**

// an associative array of strings indexed by 2-state integers,
// default is "hello".

```
string words [int] = '{default: "hello"};
```

// an associative array of 4-state integers indexed by strings, default is -1

```
integer tab [string] = '{"Peter":20, "Paul":22, "Mary":23, default:-1};
```

Queues

- **variable-size unpacked array**
- **supports constant-time insertion and removal at the beginning or the end of the array**
- **constant-time access to all its elements**
- **Syntax: <type> <q var name> [\$ [:constant]]**

`byte q1[$];` // A queue of bytes

`string names[$] = {"Bob"};` // A queue of strings with one element

`integer Q[$] = {3, 2, 7};` // An initialized queue of integers

`bit q2[$:255];` // A queue whose maximum size is 256 bits

- 0 represents first element of queue and \$ last element

`item = q1[0];` // read leftmost (first) item from list

`item = q1[$];` // read rightmost (last) item from list

`q1 = q1[1:$];` // delete leftmost (first) item of q1

`q1 = q1[0:$-1];` // delete rightmost (last) item of q1

`q1 = {};` // clear the q1 list

Queue methods

function int **size**()

- Returns the number of items in the queue. If the queue is empty, it returns 0.

function void **insert** (int index, queue_type item)

- Inserts the given item at the specified index position
- $Q.insert(i, e) \Rightarrow Q = \{Q[0:i-1], e, Q[i, \$]\}$

function void **delete** (int index)

- Deletes the item at the specified index position. If index is not specified delete all
- $Q.delete(i) \Rightarrow Q = \{Q[0:i-1], Q[i+1, \$]\}$
- $Q.delete() \Rightarrow Q$ is empty

function queue_type **pop_front**()

- Removes and returns the first element of the queue
- $e = Q.pop_front() \Rightarrow e = Q[0]; Q = Q[1, \$]$

function queue_type **pop_back**()

- Removes and returns the last element of the queue
- $e = Q.pop_back() \Rightarrow e = Q[\$]; Q = Q[0, \$-1]$

function void **push_front** (queue_type item);

- Inserts the given element at the front of the queue
- $Q.push_front(e) \Rightarrow Q = \{e, Q\}$

function void **push_back** (queue_type item);

- Inserts the given element at the end of the queue
- $Q.push_back(e) \Rightarrow Q = \{Q, e\}$

Prompt:

How to choose the Right Array Type for Aggregated Data in SV

Array manipulation methods

SV supports a variety of methods to search, order and reduce arrays

- Syntax: `expression.array_method_name [ ( list_of_arguments ) ] [ with ( expression ) ]`

- **Search**

<code>find()</code>	returns all the elements satisfying the given expression
<code>find_index()</code>	returns the indexes of all the elements satisfying the given expression
<code>find_first()</code>	returns the first element satisfying the given expression
<code>find_first_index()</code>	returns the index of the first element satisfying the given expression
<code>find_last()</code>	returns the last element satisfying the given expression
<code>find_last_index()</code>	returns the index of the last element satisfying the given expression
<code>min()</code>	returns the element with the minimum value
<code>max()</code>	returns the element with the maximum value
<code>unique()</code>	returns all elements with unique values
<code>unique_index()</code>	returns the indexes of all elements with unique values

```
string SA[10], qs[$];  
int IA[*], qi[$];  
qi = IA.find( x ) with ( x > 5 );           // Find all items greater than 5  
qs = SA.unique( s ) with ( s.toLowerCase ); // Find all unique lowercase strings
```

Array manipulation methods...

- **Order**

- `reverse()` reverses all the elements of the array (packed or unpacked)
- `sort()` sorts the unpacked array in ascending order
- `rsort()` sorts the unpacked array in descending order
- `shuffle()` randomizes the order of the elements in the array

```
string s[] = { "teacher", "apple", "student" };  
s.reverse(); // s becomes { "student", "apple", "teacher" };  
struct { byte red, green, blue; } video [512];  
video.sort() with ( item.red ); // sort video using the red byte only  
video.sort( sel ) with ( sel.blue << 8 + sel.green ); // sort by blue-green algorithm
```

- **Reduce**

- `sum()` returns the sum of all the array elements
- `product()` returns the product of all the array elements
- `and()` returns the bitwise AND (&) of all the array elements
- `or()` returns the bitwise OR (|) of all the array elements
- `xor()` returns the logical XOR (^) of all the array elements

```
byte b[] = { 1, 2, 3, 4 };  
int y;  
y = b.sum() ; // y becomes 10 => 1 + 2 + 3 + 4  
y = b.product() ; // y becomes 24 => 1 * 2 * 3 * 4
```

Array querying methods

SystemVerilog provides system functions to return information about an array

- \$bits, \$left, \$right, \$low, \$high, \$increment, \$size, \$dimensions
- Return type is `integer` for all querying functions
- `$dimensions( array_id )`: Returns the number of dimensions in the array (0 if scalar)

```
bit [3:0] [7:0] aa;
```

```
initial $display( $dimensions(aa) ); // prints " 2 "
```

- `$bits(array_id)`: Returns the number of bits in array or struct

```
initial $display( $bits(aa) ); // prints " 32 "
```

- `${left|right}(array, N = 1)`:

Returns bounds of a dimension // \$left => msb, \$right => lsb

```
bit [3:1] [7:4] cc [2:0];
```

```
initial $display( $left(cc) , $right(cc) ); // prints " 2 0 "
```

- `$increment(array_id, N = 1)`:

Returns 1 if \$left is greater than or equal to \$right

Returns -1 if \$left is less than \$right

- `$size(array_id, N = 1)`: returns number of elements in the dimension (\$high - \$low + 1) (**was \$length (now deprecated)**)

```
        3  4  1  2
bit  [7:0][5:0] e [3:0][31:0];
initial
    $display( $bits(e), $dimensions(e), $size(e,1), $size(e,2), $size(e,3), $size(e,4));
```

6144	4	4	32	8	6
------	---	---	----	---	---

Exercise: Simple FIFO

Design a Simple FIFO (First In First Out) Buffer in SystemVerilog.

The FIFO should be able to store up to 256 entries of 32bit data.

The design must handle overflow, underflow, and provide status using \$display when error condition or accessed.

Hint: decide on aggregated data type first.

Try first and then pass the problem to AI prompt. Compare your solution with AI's and learn like AI

Structure

- **A structure represents a collection of SV data types**

```
struct{  
    bit[7:0] my_byte;  
    int my_data;  
    char my_char;  
} my_struct;      // my_byte, my_data and my_char are "members" of my_struct
```

- **can be referenced as a whole**
- **individual data types that make up the structure can be referenced by name**

```
initial begin  
    my_struct.my_byte = 8'h05;  
    my_struct = {0, 99, "A"};  
end
```

Structures can:

- be packed or unpacked.(default unpacked)
- be assigned as a whole
- pass to/from a function or task as a whole
- contain arrays

Packed vs unpacked structures

Packed

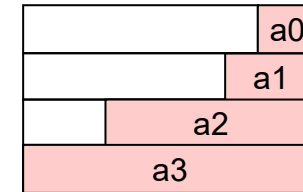
```
struct packed {  
    bit[1:0] a0;  
    bit[2:0] a1;  
    bit[5:0] a2;  
    bit[8:0] a3;  
} p_pkt;
```



- Much more useful in hardware
- Easily converted to bit-vectors
- May be accessed as a whole
- First member specified is most significant
- Can be declared as signed/unsigned for arithmetic
- Limitations
 - Only integer types (bit, logic, int, reg, time)
 - Unpacked arrays/structures NOT allowed

Unpacked

```
struct {  
    bit[1:0] a0;  
    bit[2:0] a1;  
    bit[5:0] a2;  
    bit[8:0] a3;  
} u_pkt;
```



- Default for structs
- Tool dependent implementation.
- Think of as a logical grouping of member elements
- Each member may be assigned differently

Usage of structures

- **Structures are ideal way to encapsulate data "packets"**
- **Use as data or control abstraction in architectural models**
- **Use as abstraction for top-down I/O design**
- Behavioral:
 - pass structures through ports, as arguments to tasks/functions etc.
 - Use to refine structure size/content
- RTL:
 - break structure apart & define final I/O
- Verification:
 - to encapsulate constraint/randomization weights

Unions

- **A union is a data type that represents a single piece of storage that can be accessed using one of the named member data types (same as C)**
 - Can be both packed and unpacked.
 - Memory footprint is the size of the largest member
 - Used to reduce memory consumption of a design
 - All members begin at the same memory address
 - The member read must also be the most recently written

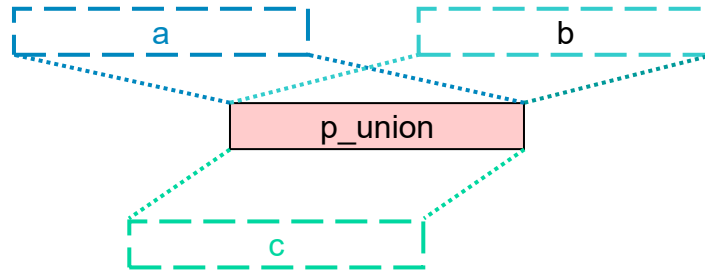
```
typedef union {int i; shortreal f;} num; // named union type
num n;
n.f = 0.0; // set n in floating-point format
typedef struct {
    bit isfloat;
    union {int i; shortreal f;} n; // anonymous union type
} tagged_st; // named structure
```

Packed Union

A packed union contains 1 or more packed members

- All members are the same size and occupy the same space
- Data may be written via one member, read by another
- Unlike C/C++ SV unions are independent of platform byte ordering
- The union may be accessed as a whole

```
union packed {  
    int    a;  
    integer b;  
    reg[31:0] c;  
} p_union;
```



Characteristics

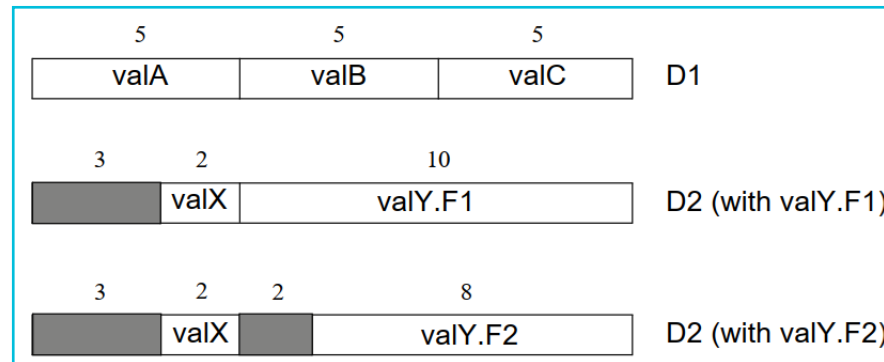
- Can be soft packed or hard packed (default: hard)
- Easy conversion to bit-vectors
- May be accessed as a whole
- First member specified is most significant i.e. last member is LSB
- May be declared as signed for arithmetic
- Non-integer datatypes (e.g. real) are NOT allowed
- Unpacked arrays/structures are NOT allowed
- *if any member is 4-state, all members are cast to 4-state*

Soft packed union

soft qualifier is used to indicate the soft packing: **union soft packed {**

- When union is soft packed, members of that union do not have to be of the same size
- size is equal to the number of bits needed to represent the maximum of the sizes of the members.
- bits of each member are right-justified [i.e., towards the least significant bits (LSBs)].

```
typedef union soft packed {  
    struct packed {  
        bit [4:0] valA, valB, valC;  
    } D1;  
    struct packed {  
        bit[1:0] valX;  
        union soft {  
            bit [9:0] F1;  
            bit [7:0] F2;  
        } valY;  
    } D2;  
} Data_u;
```



Paper and Pen Exercise: type conversion

Problem:

Analyze the following code and predict the output. Explain what happens at each assignment and whether data is lost or transformed:

```
bit [7:0] a = 8'hFF;  
int b;  
logic [7:0] c;  
shortint d;
```

```
b = a;           // What is b?  
c = 8'bxxxx_1010; // What is c?  
a = c;           // What is a now?  
d = 16'h8000;    // What is d in decimal?  
a = d;           // What is a now?
```

For each assignment:

- State the final value in hex/binary/decimal
- Explain any type conversion that occurred
- Identify if any data/information was lost

Paper and Pen Exercise: debug the data type issues

Problem:

The following code has 6 data type related bugs. Find them, explain why each is a problem, and provide the corrected code:

```
integer counter = 0; // Bug 1?  
bit [7:0] data = 8'bxxxx_0000; // Bug 2?
```

```
class packet;  
    bit [31:0] address;  
    byte payload = 300; // Bug 3?  
    string msg;  
endclass
```

```
function int calculate(bit [15:0] x, y); // Bug 4?  
    int result;  
    result = x * y; // Bug 5?  
    return result;  
endfunction
```

```
real delay = 10; // Bug 6?
```

For each bug: Explain the issue and provide the fix

The background features a gradient from dark blue at the top to teal at the bottom. A trail of glowing, out-of-focus particles in shades of teal and light blue curves from the top right towards the bottom right corner.

Questions ?

SIEMENS